



Morpho: MidnightBundlesV2 - 4f7ea16 Security Review

Auditors

MiloTruck, Lead Security Researcher

Saw-mon and Natalie, Lead Security Researcher

Om Parikh, Security Researcher

Report prepared by: Lucas Goiriz

October 6, 2026

Contents

1	About Spearbit	2
2	Introduction	2
3	Risk classification	2
3.1	Impact	2
3.2	Likelihood	2
3.3	Action required for severity levels	2
4	Executive Summary	3
4.1	Scope	3
5	Findings	4
5.1	Low Risk	4
5.1.1	Zero-value transfers revert <code>MidnightBundlesV2</code> operations for nonstandard tokens	4
5.1.2	Ratifier Midnight instance is not checked	5
5.1.3	Stale sell-all credit targets revert or open debt	6
5.1.4	Stale callback liquidity causes executable partial fills to be skipped	6
5.1.5	Settlement-fee changes can revert bundles before offer skipping	7
5.1.6	Withdrawal in sell-side functions can be forced to have zero liquidity	8
5.1.7	Uncaught reachable panics outside of <code>take</code> call	9
5.2	Gas Optimization	10
5.2.1	Call <code>repay</code> and <code>withdraw</code> only when positive units are provided	10
5.2.2	Fetch and set <code>MIDNIGHT</code> and <code>BLUE</code> from <code>_blueBuyCallbackFactory</code>	10
5.2.3	Hoist continuous fee cap check out of the offer fill loop	11
5.3	Informational	11
5.3.1	Redundant continuous fee checks on sell paths	11
5.3.2	Midnight taker fills leave the user's offer groups unconsumed	12
5.3.3	Minor Issues	12
5.3.4	Small fills can revert cancellation and reposting	13
5.3.5	<code>midnightBundlesV2CancelAndMake()</code> cannot be called for a new <code>blueMarket</code>	13
6	Appendix	15
6.1	<code>MidnightBundlesV2</code> implementation changes	15

1 About Spearbit

Spearbit is a decentralized network of expert security engineers offering reviews and other security related services to Web3 projects with the goal of creating a stronger ecosystem. Our network has experience on every part of the blockchain technology stack, including but not limited to protocol design, smart contracts and the Solidity compiler. Spearbit brings in untapped security talent by enabling expert freelance auditors seeking flexibility to work on interesting projects together.

Learn more about us at spearbit.com

2 Introduction

`MidnightBundlesV2` contract provides end-user entrypoints for the Midnight fixed-rate lending protocol. It iterates on the `MidnightBundlesV1` contract. On the maker side, a new entrypoint lets makers update their offers in a single transaction: cancel existing offers, fund new ones with collateral or with loan assets supplied to Morpho Blue, and ratify and publish a new set of offers. On the taker side, sell functions first withdraw available credit before taking offers, and buy functions can repay remaining debt after taking offers, allowing positions to be fully exited. Finally, all functions now support native tokens and a deadline, while permit-based approvals are removed.

Disclaimer: This security review does not guarantee against a hack. It is a snapshot in time of Morpho: Midnight-BundlesV2 - 4f7ea16 according to the specific commit. Any modifications to the code will require a new security review.

3 Risk classification

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

3.1 Impact

- High - leads to a loss of a significant portion (>10%) of assets in the protocol, or significant harm to a majority of users.
- Medium - global losses <10% or losses to only a subset of users, but still unacceptable.
- Low - losses will be annoying but bearable--applies to things like griefing attacks that can be easily repaired or even gas inefficiencies.

3.2 Likelihood

- High - almost certain to happen, easy to perform, or not easy but highly incentivized
- Medium - only conditionally possible or incentivized, but still relatively likely
- Low - requires stars to align, or little-to-no incentive

3.3 Action required for severity levels

- Critical - Must fix as soon as possible (if already deployed)
- High - Must fix (before deployment if not already deployed)
- Medium - Should fix
- Low - Could fix

4 Executive Summary

Over the course of 2.5 days in total, [Morpho](#) engaged with [Spearbit](#) to review the [bundles](#) protocol. In this period of time a total of **15** issues were found.

Summary

Project Name	Morpho
Repository	bundles
Review Commit	4f7ea167
Final Commit	2ed954e5
Type of Project	DeFi, Lending
Audit Timeline	Sep 23rd to Sep 25th

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	0	0	0
Low Risk	7	2	5
Gas Optimizations	3	1	2
Informational	5	1	4
Total	15	4	11

The Spearbit team reviewed Morpho's [bundles](#) holistically on commit hash [2ed954e5](#) and concluded that all findings were addressed and no new vulnerabilities were identified.

4.1 Scope

The security review had the following components in scope for [bundles](#) on commit hash [4f7ea167](#):

```
src/midnight
├─ MidnightBundlesV2.sol
├─ interfaces
│   └─ IMidnightBundlesV2.sol
```

5 Findings

5.1 Low Risk

5.1.1 Zero-value transfers revert `MidnightBundlesV2` operations for nonstandard tokens

Severity: Low Risk.

Context: (No context files were provided by the reviewer).

Description: Let $w = \min(t, c, \ell)$ be the credit withdrawn before selling, where t is the target, c the caller's credit, and ℓ the market's withdrawable liquidity. Both `MidnightBundlesV2` sell entrypoints call `MIDNIGHT.withdraw(..., w, ...)` unconditionally. For a borrower with no credit, $c = 0$ and hence $w = 0$, even when the sell target is positive and valid offers are available. Midnight's `withdraw` forwards this amount to the loan token. A token rejecting zero transfers therefore reverts the bundle before any offer is taken.

The buy entrypoints likewise call `MIDNIGHT.repay` whenever `repayEnabled` is true, even if offers completely filled the target or the caller has no debt. The computed repayment is zero; Midnight's `repay` forwards it to `transferFrom`, reverting the entire bundle, including successful prior fills. These calls are outside the offer loop's `try / catch`.

Additional zero-value calls occur when the buy input (`maxBuyerAssets` or `targetBuyerAssets`) is zero, when a sell payout is zero, or when an entry in `collateralSupplies` has zero assets. The maker and both sell entrypoints pull each collateral entry without a zero check. Unlike a zero initial credit withdrawal, callers can avoid zero collateral entries by omitting them.

The consequence is failed bundle execution for the affected token and input combinations; no fund loss or failure of all direct Midnight operations is established.

i Note:

ERC-20 requires zero-value transfers to be treated as normal transfers and emit `Transfer`. The following deployments violate that requirement.

Token	Contract address / source	Chain (ID)	Market cap (USD)	Result for both methods
BNB, legacy ERC-20	<code>0xb8c77482e45f1f44de1745f52c74426c631bdd52</code>	Ethereum (1)	\$103,583,805,704 (BNB-wide)	Reverts without a reason string; source explicitly rejects <code>_value <= 0</code> .
ETHLend / LEND, legacy token	<code>0x80fb784b7ed66730e8b1dbd9820afd29931aab03</code>	Ethereum (1)	N/A (reported supply unavailable)	Reverts without a reason string; also documented in weird-erc20 .
HOGES	<code>0xfad45e47083e4607302aa43c65fb3106f1cd7607</code>	Ethereum (1)	\$1,381,564	Transfer amount must be greater than zero
FLOKI	<code>0xcfc0c122c6b73ff809c693db761e7bae6e2b6a2e</code>	Ethereum (1)	\$275,799,677	FLOKI: <code>_transfer:ZERO_AMOUNT</code> : Transfer amount must be greater than zero.

Baby Doge Coin / BabyDoge	<code>0xc748673057861a797275cd8a068abb95a902e8de</code>	BNB Smart Chain (56), BEP-20	\$76,854,601	Transfer amount must be greater than zero
SPX6900 / SPX	<code>0xe0f63a424a4439cbe457d80e4f4b51ad25b2c56c</code>	Ethereum (1)	\$408,965,974	Transfer amount must be greater than zero
Liquid Staked ETH / LsETH	<code>0x8c1bed5b9a0928467c9b1341da1d7bd5e10b6549</code>	Ethereum (1)	\$847,024,814	<code>NullTransfer()</code> (<code>0xdac85b6c</code>)
NGI+	<code>0xf252c5bd43907a6cab079e990845a37a7c5730d9</code>	Ethereum (1)	N/A (CoinGecko contract lookup unavailable)	Invalid amount ; source requires amount > 0 .

Collateral token	Chain	Dated markets	Loan token	Zero-transfer condition
FLOKI	Ethereum	5	USDC	Zero amount rejected
SPX6900	Ethereum	5	USDC	Zero amount rejected
LsETH	Ethereum	5	USDC	Zero amount rejected
NGI+	Ethereum	5	USDC	Zero amount rejected

- View these fixed rate markets [here](#).

Recommendation: In `MidnightBundlesV2` , call `MIDNIGHT.withdraw` only when `withdrawUnits > 0` , and call `MIDNIGHT.repay` only when the computed repayment is positive. Skip zero-value direct pulls and payouts, and omit zero-asset collateral supply operations. Preserve target, slippage, and authorization requirements. Add regression coverage with a token that rejects zero transfers for selling without existing credit and buying with `repay Enabled` when offers fully fill the target.

Morpho: We document that bundles `inherits` token requirements from midnight and blue. `Midnight excludes` tokens reverting on zero-transfer.

Speabit: But some of the Morpho listed markets use tokens that break those assumptions.

Morpho: Thanks for noticing, we will flag it to the engineering team. Even if we add checks here, there are no such checks in midnight itself, hence does not fully solve the issue. We prefer bundles to work under the assumption of standard ERC-20 tokens.

Spearbit: This is partially fixed in the final commit `07f293b383824b35adc745454ac2622a27eb0502` for the `repay` and `withdraw` endpoints to only calls those endpoints when the amount provided is non-zero.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.2 Ratifier Midnight instance is not checked

Severity: Low Risk.

Context: [MidnightBundlesV2.sol#L83](#).

Description: Let M be the bundle's `MIDNIGHT` and R the supplied `ratifier`. For a nonzero `newRoot`, `midnightBundlesV2CancelAndMake` authorizes R on M without checking `R.MIDNIGHT() == M`. The root setter's success value does not establish this equality.

Suppose R is a supported ratifier configured for another instance M' . If the maker has also authorized the bundle on M' , root registration succeeds. Operators authorized by the maker on M' can then register further roots in R , including offers executable on M : the ratifier's `isRatified` does not bind execution to its configured instance. Thus permissions on M' can control offers on M without corresponding operator authorization on M .

Without bundle authorization on M' , the root setter instead reverts and rolls back the transaction. Exploitation therefore requires both the mismatched ratifier selection and the stated cross-instance authorization; an arbitrary caller cannot select a ratifier for another maker through this bundle.

Recommendation: Expose `MIDNIGHT()` in `IRatifiersV1Common` and, inside the nonzero-root branch before granting authorization, require `IRatifiersV1Common(ratifier).MIDNIGHT() == MIDNIGHT`, reverting with `InconsistentMidnight()` otherwise.

Morpho: Acknowledged. The natspec [here](#) already mentions that users should verify the ratifier (includes the right midnight instance). Further, in the Ratifier contract, the natspec mentions that ratifier *should only be used* with only `MIDNIGHT` instance.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.3 Stale sell-all credit targets revert or open debt

Severity: Low Risk.

Context: [MidnightBundlesV2.sol#L264-L290](#), [MidnightBundlesV2.sol#L418-L448](#).

Description: Let C be the caller's credit when quoting a full exit through `midnightBundlesV2SupplyCollateralAndSellWithUnitsTarget`, with `targetUnits = C`. Before execution, a liquidation realizing bad debt reduces the caller's effective credit to $C' < C$ through market-wide loss sharing. Continuous-fee accrual can produce the same change without an intervening transaction.

The bundle reads the updated credit but retains the fixed target. With `reduceOnly = true`, selling the remaining credit cannot satisfy `filledUnits = C`: execution reverts with `NotReduceOnly` or `OutOfOffers`. With `reduceOnly = false`, sufficient collateral, eligible offers, and execution before maturity, the bundle can instead create debt of $C - C'$ to complete the target. The existing fee-accrual test explicitly asserts this debt creation.

The assets-target sell function likewise retains a fixed output after credit decreases. Debt creation is permitted by `reduceOnly = false`; the limitation is that neither entrypoint expresses a full-credit exit using the execution-time balance.

Recommendation: Add a `type(uint256).max` sentinel for `targetUnits`, resolved to the credit returned by `updatePositionView` before withdrawal and offer execution. Retain `minSellerAssets` as the caller's output bound. Direct full-credit exits to this mode rather than a quoted exact-assets target.

Morpho: Fixed in commit [2ed954e5](#).

Spearbit: Fix verified.

5.1.4 Stale callback liquidity causes executable partial fills to be skipped

Severity: Low Risk.

Context: [MidnightBundlesV2.sol#L274-L290](#), [MidnightBundlesV2.sol#L428-L448](#).

Description: Let a sell bundle target 100 units, with quoted fills of 100 units from offer *A* and 1 unit from offer *B*. Assume unit prices, zero fees, no initial withdrawable credit, and sufficient collateral. Offer *A* is funded by a Blue callback; *B* has independent funding.

Before execution, another user borrows or withdraws Blue liquidity, reducing *A*'s executable size to 99 units without changing its consumption cap. The bundle sizes each take from the remaining target, `fill.units`, and remaining offer consumption. It does not cap the take against the callback's live funding bound.

The 100-unit take from *A* therefore reverts during the callback's Blue withdrawal. The bundle catches the revert and skips *A* entirely. It fills 1 unit from *B*, then reverts with `OutOfOffers`, although the executable partial fills $99 + 1$ meet the target. Both sell entrypoints use this sizing pattern. The callback already exposes `buyerAssetsBound` to address stale off-chain routing amounts.

Recommendation: For supported funding callbacks, query the execution-time asset bound and convert it conservatively to a unit cap before taking the offer. Include this cap in take sizing, retaining the existing handling of failed takes and final user bounds.

Morpho: Fixed in commit [07f293b3](#).

Spearbit: Fixed in [07f293b3](#) by adding the following constraints when deriving the `unitsToTake` in the taker sell endpoints:

```
TakeAmountsLib.buyerAssetsToUnits(MIDNIGHT, id, fill.offer, buyerAssetsBound(id, fill.offer))
```

where the auxiliary `buyerAssetsBound` function is:

```
/// @dev Returns the maker's callback funding cap in buyer assets for a buy offer.
/// @dev Buy offers already limit buyer assets to uint128.max through maxAssets or maxUnits and
    ↳ buyerPrice <= WAD.
function buyerAssetsBound(bytes32 id, Offer memory offer) internal view returns (uint256) {
    if (offer.callback == address(0)) return type(uint128).max;
    try IBuyerAssetsBound(offer.callback)
        .buyerAssetsBound(id, offer.market, offer.maker, offer.callbackData) returns (
            uint256 bound
        ) {
        return UtilsLib.min(bound, type(uint128).max);
    } catch {
        return type(uint128).max;
    }
}
```

Moreover, the following constraint has also been added to avoid skipping reduce only offers that might have been skipped due to the units provided being greater than the current maker's live debt:

```
fill.offer.reduceOnly ? debt(id, fill.offer.maker) : type(uint256).max
```

5.1.5 Settlement-fee changes can revert bundles before offer skipping

Severity: Low Risk.

Context: [MidnightBundlesV2.sol#L192](#), [MidnightBundlesV2.sol#L275](#), [MidnightBundlesV2.sol#L340-L342](#), [MidnightBundlesV2.sol#L344](#), [MidnightBundlesV2.sol#L429-L431](#), [MidnightBundlesV2.sol#L433](#).

Description: Let p be a buy offer's price and f the settlement fee. An assets-target sell bundle includes this offer while $p > f$. Before execution, the authorized fee setter raises the applicable fee to $f' > p$, within the protocol's fee limits. Assume later offers can satisfy the target at the new fee.

`sellerAssetsToUnits` computes $p - f'$ while sizing the first take. This subtraction underflows outside the `try/catch` around `Midnight.take`, reverting the entire bundle instead of skipping the invalid offer. At $p = f'$, the conversion instead divides by zero.

The assets-target buy function has the analogous failure when a sell offer's price plus the updated settlement fee exceeds `WAD : buyerAssetsToUnits` reverts outside `try/catch`, preventing fallback to later offers. These scenarios require an applicable fee change; arbitrary callers cannot invoke the fee setter.

Recommendation: Make offer sizing return a non-executable result for invalid live prices and skip those offers before conversion. In particular, skip seller conversions with nonpositive seller price and buyer conversions with buyer price above `WAD`. Apply the same handling to conversions reached through `consumableUnits`.

Another design method would be to move inner blocks of the offer fill loops into a `public` function in `MidnightBundlesV2` where this contract self-calls into in a `try/catch` block, thus skipping all possible ways that an offer fill might make the execution revert.

Morpho: We acknowledge this issue. Note that it was pre-existing on the midnight bundles v1, and that this is a liveness issue that can be mitigated by changing the arguments of the functions (lowering the fill amount, or removing offers).

To give more details, here are the scenarios in which the `TakeAmountsLib` functions could revert when called in the bundles:

1. If `buyerPrice > WAD` in `buyerAssetsToUnits`. Note that this can't happen in the case where the buyer is the maker, because the `tickToPrice` function never gives prices $> WAD$. So we consider the case where the taker is the buyer. But the taker would probably prefer that the bundles revert in that case: it makes no sense for buying new credit (negative rate) and neither for repaying debt (always possible to have something better with `repayEnabled`). Prices too close to `WAD` for the taker-buyer should be filtered out by the router.
2. `offer.buy` and `settlementFee >= offerPrice`. Then it reverts for underflow or potentially division by 0. Note that if the offers are sorted by price then as soon as `settlementFee >= offerPrice` for one offer, that stays true for following offers. So even if the iteration is skipped, the function would eventually revert. Also it would mean that the taker is selling at a loss for these offers. Prices too close to 0 for the taker-seller should be filtered out by the router.
3. Price is 0 which produces a division by 0. Because of case 2, we can restrict ourselves to `!offer.buy`. In that case the offer price is 0 (looking at what `TakeAmountsLib` computes). The rate is infinite here, which also isn't very likely except if the maker-seller wants to transfer its credits. It's ok if this is not supported by the bundler, this can be done on `Midnight` directly if needed. Offer price 0 to sell should be filtered out by the router.

As a summary: the cases where one take-iteration reverts are bad cases for the taker, or unsupported. The router could & should take some buffer to ensure to not hit those, and the bundler reverts if the buffer is not enough.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.6 Withdrawal in sell-side functions can be forced to have zero liquidity

Severity: Low Risk.

Context: [MidnightBundlesV2.sol#L264-L268](#), [MidnightBundlesV2.sol#L418-L422](#).

Description: Both sell-side functions, `midnightBundlesV2SupplyCollateralAndSellWithUnitsTarget()` and `midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget()`, call `Midnight.withdraw()` to withdraw available credit before filling buy order.

For example, in `midnightBundlesV2SupplyCollateralAndSellWithUnitsTarget()`:

```
(uint128 takerCreditBefore,,) = IMidnight(MIDNIGHT).updatePositionView(market, id, msg.sender);
uint256 withdrawUnits = min(targetUnits, takerCreditBefore,
↪ IMidnight(MIDNIGHT).withdrawable(id));

IMidnight(MIDNIGHT).withdraw(market, withdrawUnits, msg.sender, address(this));
```

```
uint256 filledUnits = withdrawUnits;
uint256 filledSellerAssets = withdrawUnits;
```

However, an attacker can front-run a sell transaction and use up `withdrawable` in a market. For example, a buyer can call `take()` and call `withdraw()` during the `onBuy()` callback to use up `withdrawable`. This would cause the user's sell call to always withdraw zero (ie. entire target amount has to be filled via offers).

This is harmful to the user since withdrawing is always preferred to filling offers to reduce credit/increase debt. The ratio of assets to units is 1:1 during withdrawals, in contrast, when filling a buy offer, users receive less assets for the same amount of units since they incur interest and the settlement fee.

Although, note that there is no incentive for an attacker to perform this, since they would just incur the settlement fee while consuming `withdrawable`.

A similar argument can be made for the buy-side functions (ie. forcing `take()` to revert when filling offers so that `targetUnits / targetAssets` is consumed via `repay()`), but the likelihood there is much lower since:

- It doesn't make sense for makers to force `take()` to revert when filling their own offers (eg. through the maker callback). They would just cancel the offer.
- A third party (ie. not the taker or maker) could cause `take()` to revert via competing fills (eg. consuming the same offer, or an offer in the same group, first), but this should be allowed since it is a legitimate use of the protocol.

Recommendation: `minSellerAssets` and `maxUnits` seems sufficient to protect the user, since it allows the user to bound the minimum assets they are willing to receive for units (and vice versa).

Therefore, consider simply documenting this attack vector. Additionally, the user should always set `minSellerAssets / maxUnits` with the expectation that `withdrawUnits` will be zero and the entire target units/assets will be filled via offers.

Morpho: We acknowledge this issue. There is no guarantee that the withdrawable liquidity is available at execution time, neither from the bundles contract or when calling withdraw directly. This is similar to the fact that there is no guarantee that the offers passed are not taken before execution. So we feel that it is expected, and doesn't require an additional comment.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.1.7 Uncaught reachable panics outside of `take` call

Severity: Low Risk.

Context: *(No context files were provided by the reviewer).*

Description: `unitsToTake` is an argument to `take`, so it is computed before the `try`. Computing it calls `TakeAmountsLib`, which duplicates Midnight's price arithmetic including its reverts (:26). Those reverts land outside the try and abort the whole bundle.

bundle's natspec says:

```
// The buy/sell functions below skip the offer if the take reverted. This avoids reverting the
↪ whole call when other offers passed as argument still have liquidity.
```

while, there could be cases that it panics in mirroring calculations before reaching `take` reverting the whole bundle.

for e.g in `sellerAssetsToUnits`.

```

IMidnight(midnight).settlementFee(id, UtilsLib.zeroFloorSub(offer.market.maturity,
    ↳ block.timestamp));
uint256 sellerPrice = offer.buy ? offerPrice - settlementFee : offerPrice;
return
    offer.buy ? targetSellerAssets.mulDivUp(WAD, sellerPrice) :
    ↳ targetSellerAssets.mulDivDown(WAD, sellerPrice);

```

- Will panic when `settlementFee > offerPrice` when `buy = true`.
- Will panic when `sellerPrice = 0` (division-by-zero).

same is true for `buyerAssetsToUnits`.

Moreover, it reduces to condition `tickToPrice(offer.tick) < settlementFee(id, ttm)`.

The qualifying tick band is state-dependent, not fixed. `settlementFee` scales with time to maturity and with the market's configured breakpoints, so the band is widest on long-dated markets and contracts toward the bottom of the tick range as maturity approaches.

Any `feeSetter` change moves it again. An offer can cross into the band with no action by its maker, and raising a market's fee turns every low-tick offer already resting in it into the band.

Recommendation:

- Try experimenting with sizing return 0 instead of reverting from lib, also, alternate pair of functions can be created such as `consumableUnitsOrZero`, `buyerAssetsToUnitsOrZero` and `sellerAssetsToUnitsOrZero`.
- `MidnightBundlesV2TakerTest` always sets `settlementFee` to 0, so these interactions are never tested, considering testing with `settlementFee != 0` and adding regression tests to fill coverage gap.

Morpho: We acknowledge this issue. Note that it was pre-existing on the midnight bundles v1, and that this is a liveness issue that can be mitigated by changing the arguments of the functions (lowering the fill amount, or removing offers).

To give more details, here are the scenarios in which the `TakeAmountsLib` functions could revert when called in the bundles:

1. If `buyerPrice > WAD` in `buyerAssetsToUnits`. Note that this can't happen in the case where the buyer is the maker, because the `tickToPrice` function never gives prices $> WAD$. So we consider the case where the taker is the buyer. But the taker would probably prefer that the bundles revert in that case: it makes no sense for buying new credit (negative rate) and neither for repaying debt (always possible to have something better with `repayEnabled`). Prices too close to `WAD` for the taker-buyer should be filtered out by the router.
2. `offer.buy` and `settlementFee >= offerPrice`. Then it reverts for underflow or potentially division by 0. Note that if the offers are sorted by price then as soon as `settlementFee >= offerPrice` for one offer, that stays true for following offers. So even if the iteration is skipped, the function would eventually revert. Also it would mean that the taker is selling at a loss for these offers. Prices too close to 0 for the taker-seller should be filtered out by the router.
3. Price is 0 which produces a division by 0. Because of case 2, we can restrict ourselves to `!offer.buy`. In that case the offer price is 0 (looking at what `TakeAmountsLib` computes). The rate is infinite here, which also isn't very likely except if the maker-seller wants to transfer its credits. It's ok if this is not supported by the bundler, this can be done on Midnight directly if needed. Offer price 0 to sell should be filtered out by the router.

As a summary: the cases where one take-iteration reverts are bad cases for the taker, or unsupported. The router could & should take some buffer to ensure to not hit those, and the bundler reverts if the buffer is not enough.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2 Gas Optimization

5.2.1 Call `repay` and `withdraw` only when positive units are provided

Severity: Gas Optimization.

Context: [MidnightBundlesV2.sol#L205](#), [MidnightBundlesV2.sol#L266](#), [MidnightBundlesV2.sol#L358](#), [MidnightBundlesV2.sol#L420](#).

Description/Recommendation: Just like other gas-optimisations in this contract call the `repay` and `withdraw` endpoints of `MIDNIGHT` when non-zero units are provided. This might add slight gas on the paths that would require the call but it will save a lot for paths that don't.

Morpho: Fixed in commit [07f293b3](#).

Spearbit: Fix verified.

5.2.2 Fetch and set `MIDNIGHT` and `BLUE` from `_blueBuyCallbackFactory`

Severity: Gas Optimization.

Context: [MidnightBundlesV2.sol#L47-L51](#).

Description: Fetch and set `MIDNIGHT` and `BLUE` from `_blueBuyCallbackFactory`.

Recommendation:

```
constructor(address _blueBuyCallbackFactory, address _log) {
    MIDNIGHT = IBlueBuyCallbackFactory(_blueBuyCallbackFactory).MIDNIGHT();
    BLUE = IBlueBuyCallbackFactory(_blueBuyCallbackFactory).BLUE();
    BLUE_BUY_CALLBACK_FACTORY = _blueBuyCallbackFactory;
    LOG = _log;
}
```

Moreover `_blueBuyCallbackFactory` can be defined as `IBlueBuyCallbackFactory` to avoid casting from `address` to this type.

Morpho: The gas gains are negligible and only at deployment. We would rather have blue and midnight explicit and make consistency checks for readability rather than use `BlueBuyCallbackFactory` for deriving.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.2.3 Hoist continuous fee cap check out of the offer fill loop

Severity: Gas Optimization.

Context: [MidnightBundlesV2.sol#L190](#), [MidnightBundlesV2.sol#L338](#).

Description: Given an array of offer fills for the buy endpoints, if this array has more than 1 element the following same check is performed per loop iteration:

```
require(IMidnight(MIDNIGHT).continuousFee(id) <= maxContinuousFee, ContinuousFeeAboveMax());
```

Even though one would only need to perform this once and only if new atomic credits are acquired by the user. The check also gets performed for reduce only bundles.

Recommendation: Hoist continuous fee cap check out of the offer fill loop and only check this requirement when.

- The offer fill array is non-empty.
- `targetUnits` is non-zero.
- New atomic credits are acquired by the user (this might be tricky to check).

Morpho: For simplicity, we acknowledge this issue.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.3 Informational

5.3.1 Redundant continuous fee checks on sell paths

Severity: Informational.

Context: [MidnightBundlesV2.sol#L273](#), [MidnightBundlesV2.sol#L427](#).

Description: Both.

- `midnightBundlesV2SupplyCollateralAndSellWithUnitsTarget` and.
- `midnightBundlesV2SupplyCollateralAndSellWithAssetsTarget`

require `continuousFee(id) <= maxContinuousFee` before each fill. Both take buy offers, so the taker is the seller.

The current continuous fee determines the buyer's new pending fee. Seller proceeds depend on the offer price and settlement fee; selling reduces existing credit and pending fees or increases debt. Any fee accrued on the seller's existing credit derives from its recorded `pendingFee`, independently of the current continuous fee.

Thus, a market fee above `maxContinuousFee` reverts the bundle even when the maker's fee cap and the taker's asset/unit bounds permit the sale. The checks add gas without protecting the seller's execution terms.

Recommendation: Remove these checks and the corresponding `maxContinuousFee` parameters from both sell functions and their interfaces.

Morpho: Fixed in [PR 175](#).

Spearbit: Fix verified.

5.3.2 Midnight taker fills leave the user's offer groups unconsumed

Severity: Informational.

Context: [MidnightBundlesV2.sol#L159](#), [MidnightBundlesV2.sol#L234](#), [MidnightBundlesV2.sol#L304](#), [MidnightBundlesV2.sol#L385](#).

Description: Let $C = \text{consumed}[u][g]$ for a user's offer group g with `maxUnits` M and `maxAssets` $= 0$. Filling q units of the user's own offer increases C by q . Executing the same direction and quantity as a taker through `MidnightBundlesV2` leaves C unchanged: `Midnight.take` charges only `offer.maker`, and none of the four taker entrypoints accepts or updates the caller's group.

Assume $C = 0$ and the user has an outstanding, non-reduce-only sell offer for M units. The user borrows $q \leq M$ units through a sell bundle by taking another maker's buy offer. Their own sell offer remains fillable for all M units. If collateral and the other execution conditions permit, a subsequent fill raises total borrowing to $q + M$ units, although the user intended the taker execution to consume part of the same M -unit budget. With an equivalent maker fill first, only $M - q$ units would remain available. Buy offers and asset-denominated groups have the same accounting asymmetry.

This does not violate Midnight's per-offer checks: group consumption bounds maker fills, not aggregate execution across both roles. However, users combining outstanding offers with taker bundles can exceed their intended shared budget unless they separately update or cancel those offers. The taker call's quantity and slippage bounds do not constrain subsequent maker fills.

Recommendation: Consider adding a caller-owned `group` and either `maxUnits` or `maxAssets` to all four taker entrypoints. Check that the group's current consumption plus the executed amount does not exceed the supplied maximum, then atomically update `consumed[msg.sender][group]` through `Midnight.setConsumed`. Use the same denomination and asset-accounting convention as the user's maker offers so both execution paths consume the same budget.

Morpho: We acknowledge this issue. The goal of the `consumed` mapping is to constrain offers, and direct `take`s are left out of this constraint (otherwise it would have been implemented at the Midnight level). This stays true here.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.3.3 Minor Issues

Severity: Informational.

Context: [MidnightBundlesV2.sol#L5](#), [MidnightBundlesV2.sol#L19](#), [MidnightBundlesV2.sol#L109](#), [MidnightBundlesV2.sol#L216](#), [MidnightBundlesV2.sol#L255](#), [MidnightBundlesV2.sol#L370](#), [MidnightBundlesV2.sol#L406](#).

Description/Recommendation:

1. For better readability use alias for `Market` and `MarketParams` :

```
import {IMidnight, Market as MidnightMarket} from
↳ "../lib/midnight/src/interfaces/IMidnight.sol";
//...
import {IMorpho, MarketParams as BlueMarket} from
↳ "../lib/morpho-blue/src/interfaces/IMorpho.sol";
```

The above suggestion can be applied to other codebases as well when markets from different protocols are used.

2. [MidnightBundlesV2.sol#L109](#), [MidnightBundlesV2.sol#L255](#), [MidnightBundlesV2.sol#L406](#), [MidnightBundlesV2.sol#L216](#), [MidnightBundlesV2.sol#L370](#): fetching the `collateralToken` might revert due to out of bound element access.
3. **V1 review:**
 1. [Finding 16](#), item 2 still applies.
 2. [Finding 9](#) still applies.

Morpho: We acknowledge this issue:

1. The parameter name `blueMarket` already makes that explicit.
2. It is expected that it reverts.
3. For the same reasons as in v1 review.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.3.4 Small fills can revert cancellation and reposting

Severity: Informational.

Context: [MidnightBundlesV2.sol#L61-L64](#), [MidnightBundlesV2.sol#L101-L104](#).

Description: Let c be a group's consumption and m its submitted `maxConsumed`. A fill increasing consumption by $\delta > m - c$ before inclusion reverts the entire cancel-and-repost transaction, including cancellation. When $m = c$, any positive fill suffices.

The NatSpec already documents the bound, its purpose of preventing oversized replacements, and `type(uint128).max` as the opt-out. This is an intentional safety/liveness tradeoff: a taker can front-run the bundle with a small valid fill, leaving the old offers active. It is not an unconditional cancellation DoS.

Recommendation: Extend the existing NatSpec to state that exceeding the bound also reverts cancellation, leaving old offers takeable. Mention the existing cancellation-only option with the bound disabled when cancellation must proceed regardless of intervening fills.

Morpho: The consumption limit is enforced by a require before each cancellation. If any check fails, standard Solidity revert semantics roll back the entire call, including cancellations. Hence, we feel an explicit natspec is not necessary.

Morpho: Acknowledged.

Spearbit: Acknowledged.

5.3.5 `midnightBundlesV2CancelAndMake()` cannot be called for a new `blueMarket`

Severity: Informational.

Context: [MidnightBundlesV2.sol#L118-L124](#).

Description: In `midnightBundlesV2CancelAndMake()`, when `assetsToPark` is non-zero, `Blue.supply()` is called to supply assets into the specified `blueMarket`.

```
if (assetsToPark > 0) {
    address blueBuyCallback =
        IBlueBuyCallbackFactory(BLUE_BUY_CALLBACK_FACTORY).createBlueBuyCallback(msg.sender,
        ↪ callbackSalt);
    SafeTransferLib.safeTransferFrom(blueMarket.loanToken, msg.sender, address(this),
    ↪ assetsToPark);
    TokenLib.forceApproveMax(blueMarket.loanToken, BLUE);
    IMorpho(BLUE).supply(blueMarket, assetsToPark, 0, blueBuyCallback, "");
}
```

However, `supply()` reverts when `blueMarket` hasn't been created yet:

```
require(market[id].lastUpdate != 0, ErrorsLib.MARKET_NOT_CREATED);
```

So `midnightBundlesV2CancelAndMake()` cannot be called to park assets in a new blue market.

Recommendation: If this functionality is needed, consider checking if the market has been created, otherwise call `createMarket()` before `supply()`.

Morpho: Acknowledged. For the use cases we envision, users park assets in existing Blue markets. We therefore consider automatic market creation an optional convenience that is not worth supporting in this bundles.

Morpho: Acknowledged.

Spearbit: Acknowledged.

6 Appendix

6.1 MidnightBundlesV2 implementation changes

The earlier Midnight review covered commit `8de7076`. The implementation was subsequently imported into `bundles` at `2840bb7`, matching Midnight at `012e94d` except for import paths and the copyright year. Midnight later removed the contract at `d268814`.

This comparison uses the imported implementation at `2840bb7` as its baseline and `MidnightBundlesV2` at `4f7ea16` as its endpoint.

The four buy/sell paths retain ordered offer processing, per-offer fill limits, skipping reverted `take` calls, exact-target requirements, slippage bounds, and referral-fee formulas. The principal changes are:

- **Account and market selection.** Explicit `taker` and `onBehalf` parameters are removed; operations apply to `msg.sender`. An explicit `market` replaces derivation from the first offer, permitting empty offer lists where repayment, credit withdrawal, or collateral operations suffice.
- **Repayment and withdrawal.** Buy paths optionally repay the unfilled target at face value, capped by the caller's debt, after processing offers. Sell paths first withdraw credit at face value, capped by the target, updated credit, and available liquidity. Referral fees include these amounts. The standalone `repayAndWithdrawCollateral` entry point is removed; buy paths provide this operation with an empty offer list and repayment enabled.
- **Execution guards.** All entry points enforce a deadline. Trading paths check a continuous-fee cap before each take. Taker-side `reduceOnly` rejects a calculated take exceeding current debt on buys or updated credit on sells; it does not clip the take to the position.
- **Funding.** ERC-2612 and Permit2 inputs are removed from these entry points; ERC20 transfers require prior allowance. All entry points accept native tokens, wrap `msg.value`, transfer the wrapped tokens to the caller, and then perform ordinary ERC20 pulls. Wrapped-token allowance is therefore still required.
- **Maker operations.** `midnightBundlesV2CancelAndMake` checks group consumption limits, cancels groups, optionally supplies collateral, and optionally parks loan assets on Blue for a derived `BlueBuyCallback`. A nonzero root additionally grants the selected ratifier account authorization, ratifies the root directly or by signature, and publishes the supplied payload through `LOG`. The bundle does not verify correspondence between the root, payload, funding, and collateral.
- **Dependencies and interface.** Deployment adds Blue, callback-factory, and log addresses, with factory consistency checks. `Take` becomes the reordered `OfferFill`; collateral structs are unified as `CollateralTransfer`. The newer Midnight interface adds market chain/address fields, narrows offer limits to `uint128`, and adds an offer continuous-fee cap. Approval helpers move to `TokenLib`; the allowance threshold, zero-reset approval behavior, and direct calculation logic in `TakeAmountsLib` and `ConsumableUnitsLib` are retained.